

Capacity Planning & Sizing

Right-Size VMs & Optimize Host Consolidation

70% of VMs are over-provisioned on VMware. Migration is the perfect time to right-size. This guide covers CPU/memory/storage analysis, consolidation ratios, host sizing formulas, NUMA-aware placement, and growth planning. Save 30-40% on hardware by right-sizing during migration.

Right-Sizing | Consolidation Ratios | NUMA Placement | Growth Planning | Hardware Sizing | Cost Optimization

VM Right-Sizing Analysis

Most VMs use a fraction of their allocated resources — measure before migrating.

Typical Over-Provisioning (VMware Environments)

Resource	Avg Allocated	Avg Used (P95)	Waste	Right-Size Recommendation
vCPU	4 vCPUs	1.2 vCPUs	70%	Set to P95 usage + 50% headroom = 2 vCPUs (saves 50%)
Memory	8 GB	3.5 GB	56%	Set to P95 usage + 25% headroom = 4.5 GB → round to 4 GB
Storage	200 GB	45 GB	78%	Allocate actual + 50% growth = 70 GB (thin-provisioned)
Network	10 Gbps	200 Mbps	98%	VirtIO handles up to 9.5 Gbps — no action needed

Data Collection Methods

From VMware (Pre-Migration)

```
# govc: collect 30 days of performance data
govc metric.ls vm/db-server | head -20
govc metric.sample -n 720 -i 3600 \
  vm/db-server cpu.usage.average \
  mem.active.average \
  disk.usage.average \
  net.usage.average

# Or export from vCenter performance charts:
# Monitor → Performance → Advanced
# Export CSV for 30 days, hourly interval

# RVTools (free): export full VM inventory
# Produces Excel with CPU, memory, disk, network
# rvtools.com — run on Windows, connects to vCenter
```

Right-Size Calculator

```
# right-size.py — calculate optimal sizing
import json

def right_size(vm_data):
    """Calculate right-sized resources from usage data."""
    cpu_p95 = vm_data['cpu_p95_percent'] / 100
    mem_p95 = vm_data['mem_p95_gb']

    # CPU: P95 usage * allocated + 50% headroom
    optimal_cpu = max(2, round(
        cpu_p95 * vm_data['allocated_vcpus'] * 1.5
    ))

    # Memory: P95 usage + 25% headroom, round to nearest 2GB
    optimal_mem = max(2, round(
        mem_p95 * 1.25 / 2
    ) * 2)

    # Storage: actual used + 50% growth
    optimal_disk = round(
        vm_data['disk_used_gb'] * 1.5
    )

    return {
        'vcpus': optimal_cpu,
```

```
'memory_gb': optimal_mem,  
'disk_gb': optimal_disk  
}
```

Host Sizing Formulas

Calculate how many KVM hosts you need for your VM fleet.

Consolidation Ratio Guidelines

Workload Type	vCPU:pCPU	Memory	VMs/Host (typical)	Notes
General (mixed)	3:1 - 5:1	1:1 (no overcommit)	15-30	Most conservative; safe for all workloads
Web servers	5:1 - 8:1	1.2:1	25-50	Low CPU usage, can overcommit more
Application servers	3:1 - 5:1	1:1	15-25	Moderate CPU, memory-sensitive
Database servers	1:1 - 2:1	1:1 (pinned)	4-10	CPU-intensive, NUMA-aware, hugepages
VDI desktops	4:1 - 6:1	1:1	30-60	Bursty but short duration
AI/ML (GPU)	1:1	1:1	1-4	Dedicated cores + GPU passthrough

Host Sizing Calculator

Formula

```
# Number of hosts needed
Total_vCPUs = sum(vm.vcpus for vm in fleet)
Total_RAM = sum(vm.memory_gb for vm in fleet)
Total_Storage = sum(vm.disk_gb for vm in fleet)

# Per-host specs (example: Dell R760)
Host_Cores = 64      # 2x 32-core Xeon
Host_RAM = 512       # GB DDR5
Host_Storage = 15    # TB NVMe (usable)

# CPU hosts needed (with overcommit ratio)
CPU_Hosts = ceil(Total_vCPUs / (Host_Cores * Overcommit_Ratio))

# Memory hosts needed (no overcommit)
Mem_Hosts = ceil(Total_RAM / (Host_RAM * 0.90)) # 90% usable

# Storage hosts needed
Stor_Hosts = ceil(Total_Storage / Host_Storage)
```

Example: 100 VMs Fleet

- **After right-sizing:**
- Total vCPUs: 280 (was 420 before right-sizing)
- Total RAM: 640 GB (was 980 GB)
- Total Storage: 8 TB (was 18 TB, thin provisioned)
-
- **Host: Dell R760 (64 cores, 512 GB, 15 TB NVMe)**
- CPU hosts (3:1 ratio): $280 / (64 \times 3) = 1.5 \rightarrow 2$
- Memory hosts: $640 / (512 \times 0.9) = 1.4 \rightarrow 2$
- Storage hosts: $8 / 15 = 0.53 \rightarrow 1$
- Result: **2 hosts + 1 for HA = 3 hosts**
-
- **Without right-sizing: 5 hosts needed**
- **Savings: 2 hosts × \$25K = \$50K hardware**

```
# Final answer: max of all three
Hosts_Needed = max(CPU_Hosts, Mem_Hosts, Stor_Hosts)
# Add N+1 for HA
```

NUMA-Aware VM Placement

Proper NUMA placement prevents 40% performance penalties on multi-socket servers.

NUMA Architecture

```
# Typical 2-socket server NUMA topology
# Node 0: CPU 0 (cores 0-31), 256 GB local RAM, PCIe slots 1-4
# Node 1: CPU 1 (cores 32-63), 256 GB local RAM, PCIe slots 5-8
#
# Local memory access: ~80ns
# Remote memory access: ~140ns (75% slower!)
#
# Rule: Keep VM's vCPUs and memory on the SAME NUMA node

# Check NUMA topology
numactl --hardware
# node 0: cpus 0-31, size 262144 MB
# node 1: cpus 32-63, size 262144 MB
# node distances:
#   node 0 1
#     0: 10 21   ← 21 = 2.1x latency to remote node

# Bad placement (crosses NUMA boundary):
virsh vcpupin db-server 0 0    # NUMA 0
virsh vcpupin db-server 1 32   # NUMA 1 ← CROSS-NODE!

# Good placement (all on NUMA 0):
virsh vcpupin db-server 0 0
virsh vcpupin db-server 1 1
virsh vcpupin db-server 2 2
virsh vcpupin db-server 3 3
virsh numatune db-server --nodeset 0 --mode strict
```

NUMA Placement Rules

Small VMs (< 16 vCPU, < 64 GB)

- Fit within a single NUMA node
- Pin all vCPUs to cores on same NUMA node
- Pin memory to same NUMA node
- Multiple small VMs can share a NUMA node
- Max ~4 small VMs per NUMA node (avoid contention)

Large VMs (> 32 vCPU, > 256 GB)

- Must span NUMA nodes (too large for one)
- Use `numatune --mode interleave` for balanced access
- Or use vNUMA: expose NUMA topology to guest OS
- `<numa><cell cpus='0-15' memory='131072' /><cell cpus='16-31' memory='131072' /></numa>`

- Performance: optimal (100% local memory access)

- Guest OS then optimizes memory allocation per-NUMA
- Required for: large databases, in-memory analytics

Storage Capacity Planning

QCOW2 thin provisioning + compression can reduce storage needs by 60-80%.

Storage Efficiency Comparison

Method	Virtual Size	Actual Disk	Savings	Notes
VMware thick eager	100 GB	100 GB	0%	Full allocation at creation
VMware thick lazy	100 GB	100 GB	0%	Full allocation, zeroed on first write
VMware thin	100 GB	45 GB	55%	Similar to QCOW2 thin provisioning
QCOW2 (default)	100 GB	42 GB	58%	Copy-on-write, 64KB clusters
QCOW2 + compress	100 GB	25 GB	75%	zlib compression per-cluster
QCOW2 + compress + TRIM	100 GB	18 GB	82%	Guest runs fstrim to reclaim unused

Storage Sizing Example (100 VMs)

Before Right-Sizing

- Total virtual size: 20 TB (100 VMs × 200 GB avg)
- VMware thick: 20 TB physical needed
- VMware thin: 9 TB physical used
- Hardware: 4x 6.4 TB NVMe = 25.6 TB raw
- With RAID/overhead: ~19 TB usable
- **Cost: \$10,000 (4x NVMe drives)**

After Right-Sizing + QCOW2 Compression

- Total virtual size: 7 TB (right-sized + growth room)
- QCOW2 compressed: 4.2 TB physical
- With TRIM configured: 3.5 TB physical
- Hardware: 2x 3.84 TB NVMe = 7.68 TB raw
- With overhead: ~6 TB usable (70% headroom)
- **Cost: \$4,000 (2x NVMe drives)**
- **Savings: \$6,000 (60%)**

Enable TRIM/Discard in Guest VMs

Add `discard='unmap'` to libvirt disk XML and run `fstrim -av` periodically in guests (or enable continuous TRIM with `discard` mount option in `fstab`). This returns unused blocks to the QCOW2 image, keeping actual disk usage minimal. Schedule weekly `fstrim` via cron for best results.

Growth Planning & Forecasting

Plan for 2-3 years of growth to avoid emergency hardware purchases.

Growth Rate Assumptions

Resource	Conservative	Moderate	Aggressive	Based On
VM count	10%/yr	20%/yr	35%/yr	Historical VM creation rate
CPU per VM	5%/yr	10%/yr	15%/yr	Application version upgrades
Memory per VM	10%/yr	15%/yr	25%/yr	Data growth, caching, JVM heap
Storage per VM	15%/yr	25%/yr	40%/yr	Logs, databases, media files
Network	5%/yr	10%/yr	20%/yr	API traffic growth

3-Year Capacity Forecast

Moderate Growth (100 VMs start)

Year	VMs	vCPUs	RAM (GB)	Storage (TB)	Hosts
Year 0	100	280	640	3.5	3
Year 1	120	370	880	5.3	3
Year 2	144	490	1,210	8.2	4
Year 3	173	650	1,660	12.5	5

Add 1 host in Year 2, 1 more in Year 3

Hardware Procurement Timeline

- **Day 0:** 3 hosts (covers Year 0-1 with headroom)
- **Month 18:** Order 4th host (6-8 week lead time)
- **Month 30:** Order 5th host
- **Budget:** Year 0: \$75K, Year 2: \$25K, Year 3: \$25K
- **vs VMware:** Year 0: \$75K + \$180K licensing/yr
- KVM: buy hardware on demand, no license scaling
- Alert: set monitoring at 70% capacity threshold
- Procurement lead time: plan 3 months ahead

Hardware Selection Guide

Recommended server configurations for KVM hosting at different scales.

Server Recommendations

Scale	VMs	Server Model	CPU	RAM	Storage	Network	Hosts	Cost/Host
Small	10-30	Dell R660xs	1x 24-core	256 GB	4x 1.92TB NVMe	2x 25GbE	2	\$12,000
Medium	30-100	Dell R760	2x 32-core	512 GB	8x 3.84TB NVMe	2x 25GbE	3-5	\$25,000
Large	100-300	Dell R760	2x 48-core	1 TB	12x 3.84TB NVMe	2x 100GbE	8-12	\$40,000
GPU/ML	10-50	Dell R760xa	2x 32-core	512 GB	4x 3.84TB NVMe	2x 100GbE	4-8	\$35,000+GPU
Edge	5-15	Dell R360	1x 16-core	128 GB	2x 1.92TB NVMe	2x 10GbE	1-2	\$8,000

CPU Selection

- **Intel Xeon Scalable (Sapphire/Emerald Rapids):** best single-thread, proven VMware migration path
- **AMD EPYC (Genoa/Bergamo):** best core count per dollar, 96-128 cores/socket
- **Rule:** Intel for latency-sensitive, AMD for density
- Always: match CPU family across cluster for live migration
- Enable: VT-x/AMD-V, VT-d/IOMMU in BIOS

Memory Selection

- **DDR5-4800+:** required for latest Xeon/EPYC
- **ECC:** mandatory for production (error correction)
- **Populate evenly:** same DIMMs per channel per socket
- **Don't overcommit:** RAM is the #1 bottleneck for VMs
- Reserve: 10% for host OS + KVM overhead
- Hugepages: allocate 80% of host RAM as 2MB hugepages

Migration-Time Right-Sizing

Apply right-sizing during migration — the best time to change VM resources.

h2kvmctl Right-Sizing During Conversion

```
# Right-size during migration (set new resource allocation)
h2kvmctl convert \
  --source db-server.vmdk \
  --format qcow2 --compress \
  --fix-all \
  --emit-domain-xml \
  --deploy-libvirt \
  --memory 8192 \      # Was 16384 on VMware (only used 5GB)
  --vcpus 4 \          # Was 8 on VMware (only used 1.8 cores)
  --network-bridge br-db
```

```
# Batch right-sizing from analysis spreadsheet
```

```
# migration-rightsized.yaml
```

```
vms:
```

```
- name: db-server
  source: { vmdk: /data/db-server.vmdk }
  deploy:
    memory: 8192    # Right-sized from 16384
    vcpus: 4        # Right-sized from 8
    network_bridge: br-db

- name: web-server
  source: { vmdk: /data/web-server.vmdk }
  deploy:
    memory: 2048    # Right-sized from 8192
    vcpus: 2        # Right-sized from 4
    network_bridge: br0
```

Right-Sizing Safety Net

- KVM makes it easy to adjust after migration
- Add CPU: `virsh setvcpus vm 6 --config (reboot)`
- Add RAM: `virsh setmem vm 12G --config (reboot)`
- CPU hotplug: `virsh setvcpus vm 6 --live (no reboot!)`
- Memory hotplug: `virsh setmem vm 12G --live (if guest supports)`
- Start conservative, scale up if monitoring shows need

Impact of Right-Sizing (100 VMs)

- **Before:** 420 vCPUs, 980 GB RAM → 5 hosts
- **After:** 280 vCPUs, 640 GB RAM → 3 hosts
- **Hardware saved:** 2 hosts × \$25K = \$50,000
- **Power saved:** 2 hosts × 800W × 8760h × \$0.10 = \$1,400/yr
- **Rack space saved:** 2U × 2 hosts = 4U
- **Better performance:** less contention, lower latency

- Set alerts at 80% utilization for proactive scaling

- **Total 3-yr savings: \$54,200**

Sizing Summary & Checklist

Quick reference for capacity planning decisions.

Capacity Planning Checklist

#	Action	Tool/Method	Output
1	Collect 30 days of VM performance data	govc metrics, vCenter CSV, RVTools	CPU/memory/disk usage per VM (P95)
2	Calculate right-sized resources per VM	Right-sizing calculator (P95 + headroom)	Optimal vCPU, memory, disk per VM
3	Sum total resources needed	Spreadsheet or script	Total vCPUs, RAM, storage for fleet
4	Apply consolidation ratios	Workload-type based ratios	Physical CPU cores, RAM, storage needed
5	Select host configuration	Hardware selection guide (Page 6)	Server model, specs, quantity
6	Add N+1 for HA	+1 host minimum	Final host count with HA
7	Plan NUMA placement	numactl, placement rules	VM-to-NUMA-node assignment map
8	Forecast 3-year growth	Growth rate model	Procurement timeline for additional hosts
9	Set monitoring thresholds	Prometheus alerts at 70%/80%/90%	Early warning for capacity issues
10	Apply right-sizing during migration	h2kvmctl --memory --vcpus	Migrated VMs with optimal resources

Right-Size During Migration: Save 30-40% on Hardware

Migration is the one chance to fix years of VM sprawl. By right-sizing during conversion with h2kvmctl, you reduce host requirements by 40%, save \$50K+ on hardware, and get better performance through less contention. Use thin-provisioned QCOW2 with compression to cut storage needs by 75%. Plan for 3 years of growth — buy hosts incrementally as needed, not all upfront.